SOFTWARE ENGINEERING · CHAPTER 03

Lab-03: Hibernate Framework and Spring Data JPA

Rebuild the **shop** from Lab-02 on top of JPA in one project, **shop-jpa**: map Customer, Product, Order and OrderLine as entities validated against a Flyway schema, wire the associations correctly, query them with Spring Data repositories, prove how entity states, dirty checking and optimistic locking behave, then hunt down an N+1 problem with a statement counter and fix it. The challenge adds a reporting module with dynamic Criteria filters, keyset infinite scroll, pessimistic stock reservation and a 50 000-row bulk import through a `StatelessSession`.

Hibernate ORM 7.4 · Jakarta Persistence 3.2 Spring Boot 4.1.1 · Spring Data JPA JDK 25 · Maven 3.9 · H2 · Flyway

≈ 6–7 hours 5 tasks + 1 challenge

> **How to work through this lab.** The tasks build on each other inside *one* project. Commit after every task (`git tag task-N`) so the grader can see each step. Keep the SQL log on (`org.hibernate.SQL=DEBUG`) the whole time: most acceptance criteria ask you to *read what Hibernate did*, not just whether a test passed. Tick the checklist as you go (it is saved in your browser). Open hints only after 10 minutes of being stuck.

0 Setup — 20 min

1. Verify the toolchain:

```
java -version      # 25
mvn -v             # Apache Maven 3.9.x (or use ./mvnw from the generated
project)
git --version
curl --version     # or HTTPie / Postman / IntelliJ HTTP client
```

2. Generate the project at start.spring.io with these settings, or run the equivalent command:

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project -d language=java -d javaVersion=25 \
  -d bootVersion=4.1.1 -d groupId=edu.itc.se -d artifactId=shop-jpa -d
name=shop-jpa \
```

```
  -d packageName=edu.itc.shop \
  -d dependencies=data-jpa,webmvc,validation,h2,flyway \
  -o shop-jpa.zip && unzip shop-jpa.zip -d shop-jpa && cd shop-jpa
```

Dependencies: *Spring Data JPA, Spring Web MVC, Validation, H2 Database, Flyway Migration*. Spring Boot Test (JUnit 6, AssertJ, Mockito) comes automatically. Check `./mvnw dependency:tree -Dincludes=org.hibernate.orm` shows `hibernate-core:7.4.7.Final`.

3. Replace `application.properties` with `application.yml`. **Flyway owns the schema, Hibernate only validates it**, and the SQL log is on from the start:

```yaml
spring:
  datasource:
    url: jdbc:h2:mem:shop;DB_CLOSE_DELAY=-1
    username: sa
    password: ""
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
    properties:
      hibernate:
        generate_statistics: true
  h2:
    console:
      enabled: true               # http://localhost:8080/h2-console
logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.orm.jdbc.bind: TRACE
```

4. Initialise git, make the first commit, confirm `.gitignore` excludes `target/`. Create the empty folder `src/main/resources/db/migration/` (Flyway fails at start-up if the location does not exist; an empty folder is fine).

5. Target package layout (create packages as you go):

```
edu.itc.shop
├── ShopJpaApplication.java
├── domain/       Money, Address, Category, OrderStatus, Customer, Product,
Tag, Order, OrderLine
├── repository/   CustomerRepository, ProductRepository, OrderRepository,
ProductSpecs
├── service/      OrderService, StockService, ReportService, DataSeeder
(@Profile("dev"))
├── web/          OrderController, ProductController, ApiExceptionHandler
│   └── dto/      OrderSummary, ProductView, PlaceOrderRequest
└── resources/db/migration/   V1__init.sql, V2__indexes.sql
```

1 Entities and mappings, validated by Flyway

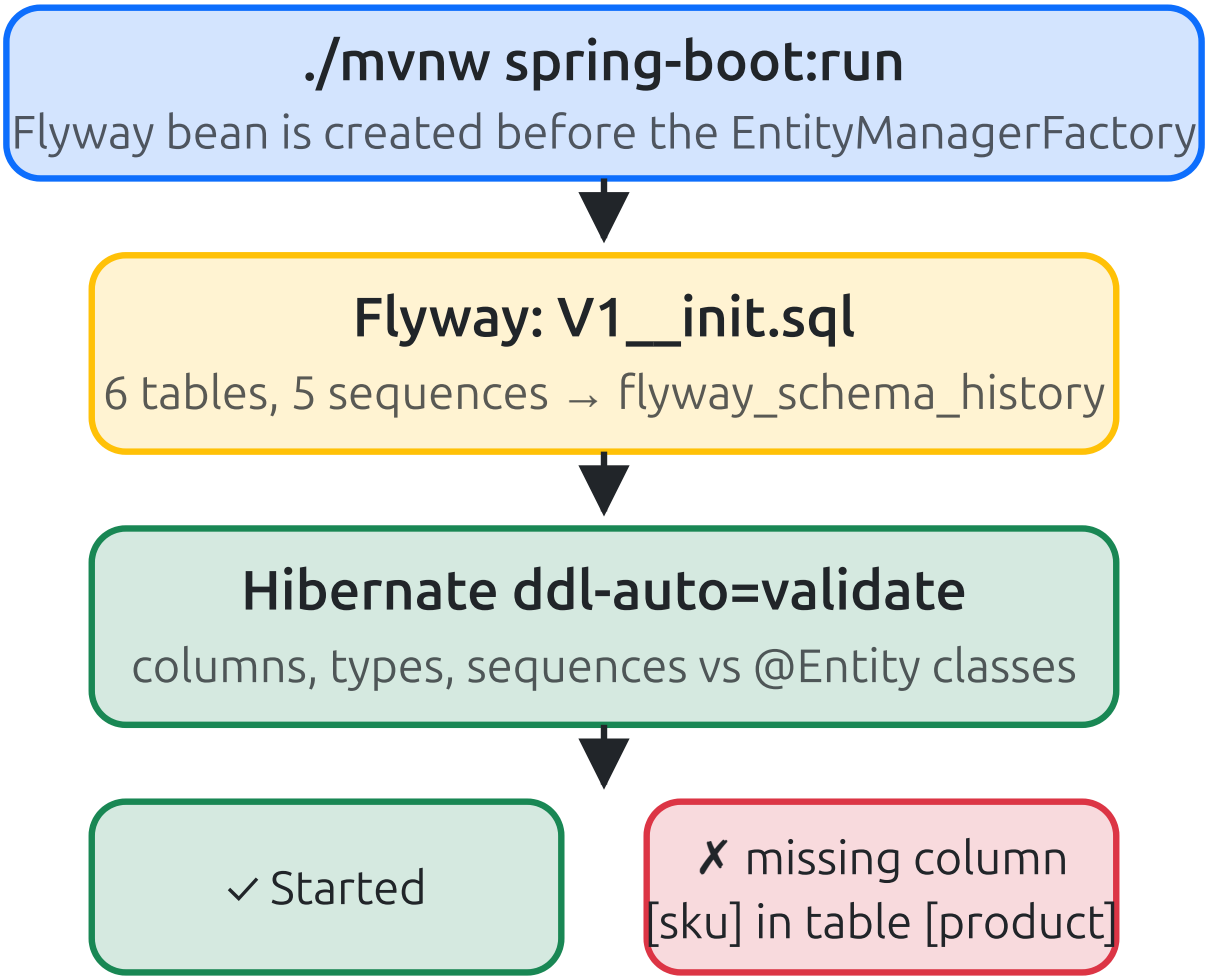
Easy 45 min

GOAL

Map the shop domain as JPA entities with embedded value objects and string enums, write the matching `V1__init.sql` migration, and let Hibernate prove at start-up that the two agree.

STEPS

1. Create the value objects as `@Embeddable` records in `domain/`: `Money(BigDecimal amount, String currency)` (columns `amount DECIMAL(12,2)`, `currency CHAR(3)`, plus a static `eur(String)` factory and `times(int)`) and `Address(String street, String city, String zip)`.
2. Create the enums `Category { BOOKS, HARDWARE, TOYS, FOOD }` and `OrderStatus { NEW, PAID, SHIPPED, CANCELLED }`. Every enum field is mapped with `@Enumerated(EnumType.STRING)`.
3. Create the entities: `Customer` (name, unique email, `@Embedded billing`, `@Embedded shipping` with `@AttributeOverride` to `ship_*` columns), `Product` (unique sku, name, `Money price`, category, stock), `Tag` (unique name), `Order` (`@Table(name = "orders")`, `@ManyToOne(fetch = LAZY) customer`, status, `Instant placedAt`, `@Version long version`) and `OrderLine` (`@ManyToOne(fetch = LAZY) order` and `product`, quantity, `Money unitPrice`). All ids: `Long`, `GenerationType.SEQUENCE` with a named generator and `allocationSize = 50`. Protected no-arg constructor, no `setId`.
4. Write `db/migration/V1__init.sql`: five sequences (`INCREMENT BY 50`), the tables `customer`, `product`, `tag`, `product_tag`, `orders`, `order_line` with primary keys, `NOT NULL`, `UNIQUE`, `CHECK (stock >= 0)`, `CHECK (quantity > 0)` and foreign keys.
5. Start the application (`./mvnw spring-boot:run`). Flyway must report *Successfully applied 1 migration* and Hibernate must start without a `SchemaManagementException`.
6. Break it on purpose: rename the column `sku` to `code` in the SQL only (keep the entity), restart, and copy the exact error message into `README.md`. Revert. Repeat once with a sequence declared `INCREMENT BY 1`; record that message too.
7. Write `ProductMappingTest` (`@DataJpaTest` + `@AutoConfigureTestDatabase(replace = Replace.NONE)` so Flyway and `validate` run in the test as well): persist a `Product` with `TestEntityManager`, `flush()`, `clear()`, reload by id and assert that `Money` and `Category` round-trip. Assert in the H2 console (or with a native query) that the `category` column contains the text `HARDWARE`, not a number.



STARTER CODE

```
package edu.itc.shop.domain;

import java.math.BigDecimal;
import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;

@Embeddable
public record Money(
        @Column(name = "amount", nullable = false, precision = 12, scale = 2)
BigDecimal amount,
        @Column(name = "currency", nullable = false, length = 3) String
currency) {

    public static Money eur(String value) { return new Money(new
BigDecimal(value), "EUR"); }

    public Money times(int qty) {
        // TODO multiply amount, keep currency
        return this;
```

```
    }
}
```

```
package edu.itc.shop.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "product")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"product_seq")
    @SequenceGenerator(name = "product_seq", sequenceName = "product_seq",
allocationSize = 50)
    private Long id;

    @Column(nullable = false, unique = true, length = 20)
    private String sku;

    @Column(nullable = false, length = 120)
    private String name;

    @Embedded
    private Money price;

    // TODO category: @Enumerated(EnumType.STRING), NOT NULL, length 20
    // TODO stock: int

    protected Product() { }

    public Product(String sku, String name, Money price, Category category) {
        // TODO
    }

    public Long getId() { return id; }
    // TODO getters; setters only for name, price, stock
}
```

```
-- src/main/resources/db/migration/V1__init.sql
CREATE SEQUENCE product_seq START WITH 1 INCREMENT BY 50;
-- TODO customer_seq, tag_seq, orders_seq, order_line_seq

CREATE TABLE product (
    id        BIGINT        PRIMARY KEY,
```

```
    sku       VARCHAR(20)   NOT NULL UNIQUE,
    name      VARCHAR(120)  NOT NULL,
    amount    DECIMAL(12,2) NOT NULL,
    currency  CHAR(3)       NOT NULL,
    category  VARCHAR(20)   NOT NULL,
    stock     INT           NOT NULL DEFAULT 0 CHECK (stock >= 0)
);
-- TODO customer (email UNIQUE, street/city/zip +
ship_street/ship_city/ship_zip)
-- TODO tag, product_tag(product_id, tag_id, PRIMARY KEY (product_id, tag_id))
-- TODO orders (customer_id FK, status, placed_at TIMESTAMP WITH TIME ZONE,
version BIGINT NOT NULL DEFAULT 0)
-- TODO order_line (order_id FK, product_id FK, quantity CHECK (quantity > 0),
amount, currency)
```

EXPECTED OUTPUT

```
INFO  o.f.c.i.s.JdbcTableSchemaHistory : Creating Schema History table
"PUBLIC"."flyway_schema_history" ...
INFO  o.f.core.internal.command.DbMigrate : Current version of schema "PUBLIC":
<< Empty Schema >>
INFO  o.f.core.internal.command.DbMigrate : Migrating schema "PUBLIC" to version
"1 - init"
INFO  o.f.core.internal.command.DbMigrate : Successfully applied 1 migration to
schema "PUBLIC", now at version v1
INFO  o.h.jpa.internal.util.LogHelper     : HHH000204: Processing
PersistenceUnitInfo [name: default]
INFO  org.hibernate.Version               : HHH000412: Hibernate ORM core
version 7.4.7.Final
INFO  o.s.b.a.o.j.JpaBaseConfiguration   : ... Initialized JPA
EntityManagerFactory for persistence unit 'default'
INFO  e.i.shop.ShopJpaApplication         : Started ShopJpaApplication in 3.1
seconds

# after renaming sku → code in V1 only:
Caused by: org.hibernate.tool.schema.spi.SchemaManagementException:
  Schema-validation: missing column [sku] in table [product]

$ ./mvnw -q test -Dtest=ProductMappingTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- H2 maps `TIMESTAMP WITH TIME ZONE` to `Instant`; plain `TIMESTAMP` fails validation with a type mismatch.

- Two embedded `Address` fields need `@AttributeOverride` on the second one, otherwise Hibernate reports a "Repeated column" error at start-up.
- `@DataJpaTest` replaces the datasource with an embedded one and sets `ddl-auto=create-drop` by default; `@AutoConfigureTestDatabase(replace = Replace.NONE)` plus `spring.jpa.hibernate.ddl-auto=validate` in the test properties keeps Flyway in charge.
- The sequence mismatch message mentions `hibernate.id.sequence.increment_size_mismatch_strategy`; do not change the strategy, fix the SQL.
- Use `TestEntityManager.persistAndGetId(p, Long.class)`; then `flush()` and `clear()` to force a real SELECT on reload.

ACCEPTANCE CHECKLIST

- [ ] All six tables and five sequences are created by `V1__init.sql`; the application starts with `ddl-auto=validate`.
- [ ] `Money` and `Address` are `@Embeddable` records; both enums use `EnumType.STRING`; no `setId` anywhere.
- [ ] Both deliberate schema mismatches (missing column, sequence increment) were reproduced and documented in `README.md`.
- [ ] `ProductMappingTest` passes against the Flyway schema and proves the enum is stored as text.

2 Associations, cascading and entity identity

Medium **50 min**

GOAL

Make `Order` the aggregate root of its lines with a correctly owned bidirectional association, add a many-to-many between products and tags, and implement entity identity so that collections and Hibernate agree.

STEPS

1. In `Order`, add `@OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true) List<OrderLine> lines`, initialised to an `ArrayList`. Expose it read-only and add the helpers `addLine(Product, int)` (creates the line, sets `line.order = this`, copies `product.getPrice()` into `unitPrice`) and `removeLine(OrderLine)`. Add `Money total()` that sums `unitPrice.times(quantity)`.
2. In `Product`, add `@ManyToMany Set<Tag> tags` with `@JoinTable(name = "product_tag", …)`; in `Tag` add the inverse `@ManyToMany(mappedBy = "tags") Set<Product> products`. Add `Product.tag(Tag)` that updates both sides.

3. Implement `equals`/`hashCode` on every entity: equal when same class and non-null equal ids; `hashCode` returns a constant per class. Implement `toString` with scalar fields only. No Lombok `@Data`/`@EqualsAndHashCode`/`@ToString` on entities.
4. Write `OrderCascadeTest` (`@DataJpaTest`, Flyway as in Task 1): persist a customer and two products, build an order with two lines through `addLine`, `em.persist(order)`, `flush()`. Read the SQL log: exactly one `insert into orders` and two `insert into order_line`, no `update`. Then `clear()`, reload, assert two lines and the total.
5. In the same test class: reload the order, call `removeLine(lines.get(0))`, `flush()`; assert that `select count(l) from OrderLine l` dropped by one (orphan removal) and that a `delete from order_line` appears in the log. Then remove the whole order and assert its remaining line is gone too (cascade REMOVE).
6. Write `EntityIdentityTest`: put a new `Product` into a `HashSet`, persist and flush it, assert `set.contains(product)` is still true; reload it in a fresh context and assert `equals` holds between the two instances while `==` does not.
7. Reproduce the classic `toString` pitfall once: temporarily include `order` in `OrderLine.toString()` and `lines` in `Order.toString()`, print an order outside a transaction, and record the exception (`StackOverflowError` or `LazyInitializationException`) in `README.md`. Revert.

STARTER CODE

```
package edu.itc.shop.domain;

import java.time.Instant;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import jakarta.persistence.*;

@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"orders_seq")
    @SequenceGenerator(name = "orders_seq", sequenceName = "orders_seq",
allocationSize = 50)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "customer_id")
    private Customer customer;
```

```
    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private OrderStatus status = OrderStatus.NEW;

    @Column(name = "placed_at", nullable = false)
    private Instant placedAt = Instant.now();

    // TODO @OneToMany(mappedBy = "order", cascade = ALL, orphanRemoval = true)
    private List<OrderLine> lines = new ArrayList<>();

    @Version
    private long version;

    protected Order() { }
    public Order(Customer customer) { this.customer = customer; }

    public OrderLine addLine(Product product, int quantity) {
        // TODO create the line with this order, product, quantity and
product.getPrice()
        // TODO add it to lines and return it
        return null;
    }

    public void removeLine(OrderLine line) {
        // TODO remove from lines and clear line.order
    }

    public List<OrderLine> getLines() { return
Collections.unmodifiableList(lines); }

    public Money total() {
        // TODO sum unitPrice.times(quantity); use Money.eur("0") for an empty
order
        return Money.eur("0");
    }

    @Override public boolean equals(Object o) {
        return this == o || (o instanceof Order other && id != null &&
id.equals(other.getId()));
    }
    @Override public int hashCode() { return Order.class.hashCode(); }
    @Override public String toString() { return "Order[id=%d,
status=%s]".formatted(id, status); }
    // getters …
}
```

EXPECTED OUTPUT

```
$ ./mvnw -q test -Dtest=OrderCascadeTest,EntityIdentityTest
DEBUG org.hibernate.SQL : select next value for orders_seq
DEBUG org.hibernate.SQL : select next value for order_line_seq
DEBUG org.hibernate.SQL : insert into orders
(customer_id,placed_at,status,version,id) values (?,?,?,?,?)
DEBUG org.hibernate.SQL : insert into order_line
(amount,currency,order_id,product_id,quantity,id) values (?,?,?,?,?,?)
DEBUG org.hibernate.SQL : insert into order_line
(amount,currency,order_id,product_id,quantity,id) values (?,?,?,?,?,?)
...
DEBUG org.hibernate.SQL : delete from order_line where id=?
...
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- If you see `insert into orders_order_line` or a start-up error about a missing table `orders_lines`, you forgot `mappedBy`: Hibernate then invents a join table.
- `order_id` is NULL in the insert when `addLine` forgets to set `line.order`; the `NOT NULL` constraint from V1 turns that into a clear failure.
- Orphan removal only triggers on the managed collection instance; do not replace `lines` with a new list (`lines = new ArrayList<>()`) or Hibernate throws *"A collection with orphan removal was no longer referenced"*.
- The many-to-many owning side is the one with `@JoinTable`; adding a tag only to `tag.products` writes nothing.
- `other.getId()` rather than `other.id`: a Hibernate proxy has an empty `id` field but its getter is intercepted.

ACCEPTANCE CHECKLIST

- [ ] `OrderLine.order` is the owning side; `Order.lines` uses `mappedBy`, cascade ALL and orphan removal; helpers keep both sides in sync.
- [ ] Persisting an order produces exactly 1 + N inserts and no updates; removing a line produces one delete (log excerpt in `README.md`).
- [ ] `Product ↔ Tag` works through `product_tag`; the collections are `Set`s.
- [ ] `equals`/`hashCode`/`toString` are hand-written on all entities and `EntityIdentityTest` passes; the `toString` pitfall is documented.

3 Repositories, queries, projections and a paginated endpoint

Medium **75 min**

GOAL

Replace hand-written data access with Spring Data repositories: derived queries, JPQL with `join fetch`, one native query, interface and record projections, a `Specification`-based search, and two REST endpoints that page and sort.

STEPS

1. Create `CustomerRepository`, `ProductRepository` and `OrderRepository` extending `JpaRepository`. Derived queries to add: `OrderRepository.findByCustomerEmailAndStatusOrderByPlacedAtDesc(String, OrderStatus)`, `countByStatus`, `findFirstByCustomerIdOrderByPlacedAtDesc`; `ProductRepository.existsBySku`, `findTop3ByCategoryOrderByPriceAmountDesc`, `findByTagsName(String)`. Start the app: every method is parsed at boot. Introduce a typo (`findByCustomerEmali…`) once and paste the boot error into `README.md`.
2. Add `@Query` JPQL: `OrderRepository.findWithLinesByCustomerEmail(String)` using `select distinct o … join fetch o.lines l join fetch l.product`, and `countLinesByStatus(OrderStatus)` with an aggregate. Add one native query `ProductRepository.findLowStock(int min)` (`SELECT p.* FROM product p WHERE p.stock < :min ORDER BY p.stock`, `nativeQuery = true`).
3. Projections in `web/dto`: interface `OrderSummary { Long getId(); OrderStatus getStatus(); Instant getPlacedAt(); CustomerName getCustomer(); interface CustomerName { String getName(); } }` returned by `Page<OrderSummary> findByStatus(OrderStatus, Pageable)`; record `ProductView(Long id, String sku, String name, BigDecimal amount, int stock)` returned by a derived `Page<ProductView> findByCategory(Category, Pageable)`. Check the SQL log: the interface projection selects only the listed columns.
4. `ProductRepository` also extends `JpaSpecificationExecutor<Product>`. Write `ProductSpecs.nameContains(String)`, `inCategory(Category)`, `cheaperThan(BigDecimal)`, `inStock()` and a static `ProductSpecs.filter(q, category, maxPrice, inStock)` that combines only the non-null ones with `Specification.allOf`.
5. Create `DataSeeder` (`@Component @Profile("dev")`, `ApplicationRunner`) that inserts 3 customers, 12 products with tags and 30 orders (2–4 lines each, mixed statuses) through the repositories inside one `@Transactional` method. Run with `--spring.profiles.active=dev`.
6. Endpoints: `GET /api/orders?status=PAID&page=0&size=5&sort=placedAt,desc` returns `Page<OrderSummary>` using a `Pageable` parameter with `@PageableDefault(size = 20, sort = "placedAt", direction = DESC)`; `GET /api/products?q=&category=&maxPrice=&inStock=&page=&size=` returns `Page<ProductView>` built from

the specification (map entities to the record in the service). Add `GET /api/customers/{email}/orders` returning full order DTOs (lines included) built from `findWithLinesByCustomerEmail` inside a `@Transactional(readOnly = true)` service method.

7. Write `OrderRepositoryTest` (`@DataJpaTest`) covering the derived query, the fetch join (assert `Hibernate.isInitialized(order.getLines())` is true after the query, false after a plain `findById` + `clear`), the native query and the projection. Save an HTTP transcript of the three endpoints in `api-session.md`.

STARTER CODE

```
package edu.itc.shop.repository;

import java.time.Instant;
import java.util.List;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import edu.itc.shop.domain.Order;
import edu.itc.shop.domain.OrderStatus;
import edu.itc.shop.web.dto.OrderSummary;

public interface OrderRepository extends JpaRepository<Order, Long> {

    List<Order> findByCustomerEmailAndStatusOrderByPlacedAtDesc(String email,
OrderStatus status);

    long countByStatus(OrderStatus status);

    Page<OrderSummary> findByStatus(OrderStatus status, Pageable pageable);

    @Query("""
        select distinct o from Order o
          join fetch o.lines l
          join fetch l.product
         where o.customer.email = :email
         order by o.placedAt desc
        """)
    List<Order> findWithLinesByCustomerEmail(@Param("email") String email);

    // TODO countLinesByStatus: select count(l) from OrderLine l where
l.order.status = :status
}
```

```
package edu.itc.shop.repository;

import java.math.BigDecimal;
import org.springframework.data.jpa.domain.Specification;
import edu.itc.shop.domain.Category;
import edu.itc.shop.domain.Money;
import edu.itc.shop.domain.Product;

public final class ProductSpecs {
    private ProductSpecs() { }

    public static Specification<Product> nameContains(String q) {
        return (root, query, cb) ->
                cb.like(cb.lower(root.<String>get("name")), "%" +
q.toLowerCase() + "%");
    }

    public static Specification<Product> cheaperThan(BigDecimal max) {
        return (root, query, cb) ->
                cb.le(root.<Money>get("price").<BigDecimal>get("amount"), max);
    }

    // TODO inCategory(Category), inStock()

    public static Specification<Product> filter(String q, Category c, BigDecimal
max, Boolean inStock) {
        return Specification.allOf(
                q == null || q.isBlank() ? null : nameContains(q),
                // TODO the other three, null when the parameter is absent
                max == null ? null : cheaperThan(max));
    }
}


@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final OrderRepository orders;
    public OrderController(OrderRepository orders) { this.orders = orders; }

    @GetMapping
    public Page<OrderSummary> list(@RequestParam OrderStatus status,
                                   @PageableDefault(size = 20, sort =
"placedAt",
                                                    direction =
Sort.Direction.DESC) Pageable pageable) {
        return orders.findByStatus(status, pageable);
```

```
    }
}
```

EXPECTED OUTPUT

```
$ curl -s "localhost:8080/api/orders?
status=PAID&page=0&size=2&sort=placedAt,desc" | jq
{
  "content": [
    { "id": 151, "status": "PAID", "placedAt": "2026-09-24T09:12:44.120Z",
"customer": { "name": "Ada Lovelace" } },
    { "id": 108, "status": "PAID", "placedAt": "2026-09-23T17:03:10.551Z",
"customer": { "name": "Grace Hopper" } }
  ],
  "page": { "size": 2, "number": 0, "totalElements": 11, "totalPages": 6 }
}

# SQL log for that request: two statements, only the projected columns
DEBUG org.hibernate.SQL : select o1_0.id,o1_0.status,o1_0.placed_at,c1_0.name
from orders o1_0
                          join customer c1_0 on c1_0.id=o1_0.customer_id where
o1_0.status=?
                          order by o1_0.placed_at desc offset ? rows fetch first
? rows only
DEBUG org.hibernate.SQL : select count(o1_0.id) from orders o1_0 where
o1_0.status=?

$ curl -s "localhost:8080/api/products?
q=key&category=HARDWARE&maxPrice=60&inStock=true" | jq '.content[].sku'
"KB-1"
"KB-7"

$ ./mvnw -q test -Dtest=OrderRepositoryTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Boot's default `Page` JSON is the stable "page" envelope (`spring.data.web.pageable.serialization-mode=via_dto` is the default in Spring Data 4); do not build your own wrapper.
- Derived queries on nested properties: `findByCustomerEmail…` walks `order.customer.email`; `findByTagsName` joins the `tags` collection. If a property is ambiguous, separate with an underscore (`Customer_Email`).
- A fetch join returns duplicates without `distinct` in older versions; Hibernate 6+ de-duplicates parents automatically, but keep `distinct` for clarity.

- Never return `Order` entities from the controller: build a record `OrderDto(id, status, placedAt, List<LineDto> lines, Money total)` inside the transactional service method.
- Interface projections with a nested interface produce a join in SQL; an open projection with `@Value` would load the whole entity instead. Check the log to see the difference.

ACCEPTANCE CHECKLIST

- [ ] Derived queries, two `@Query` JPQL methods and one native query exist and are covered by `OrderRepositoryTest`; the boot-time typo error is documented.
- [ ] Interface and record projections are used; the SQL log shows only projected columns for `/api/orders`.
- [ ] `/api/products` combines optional filters through `Specification`s and pages correctly; no entity is serialised by a controller.
- [ ] The fetch-join endpoint returns orders with lines from a `readOnly` transaction; `api-session.md` contains the three transcripts.

4

Entity states, dirty checking, transactions and optimistic locking

Hard 75 min

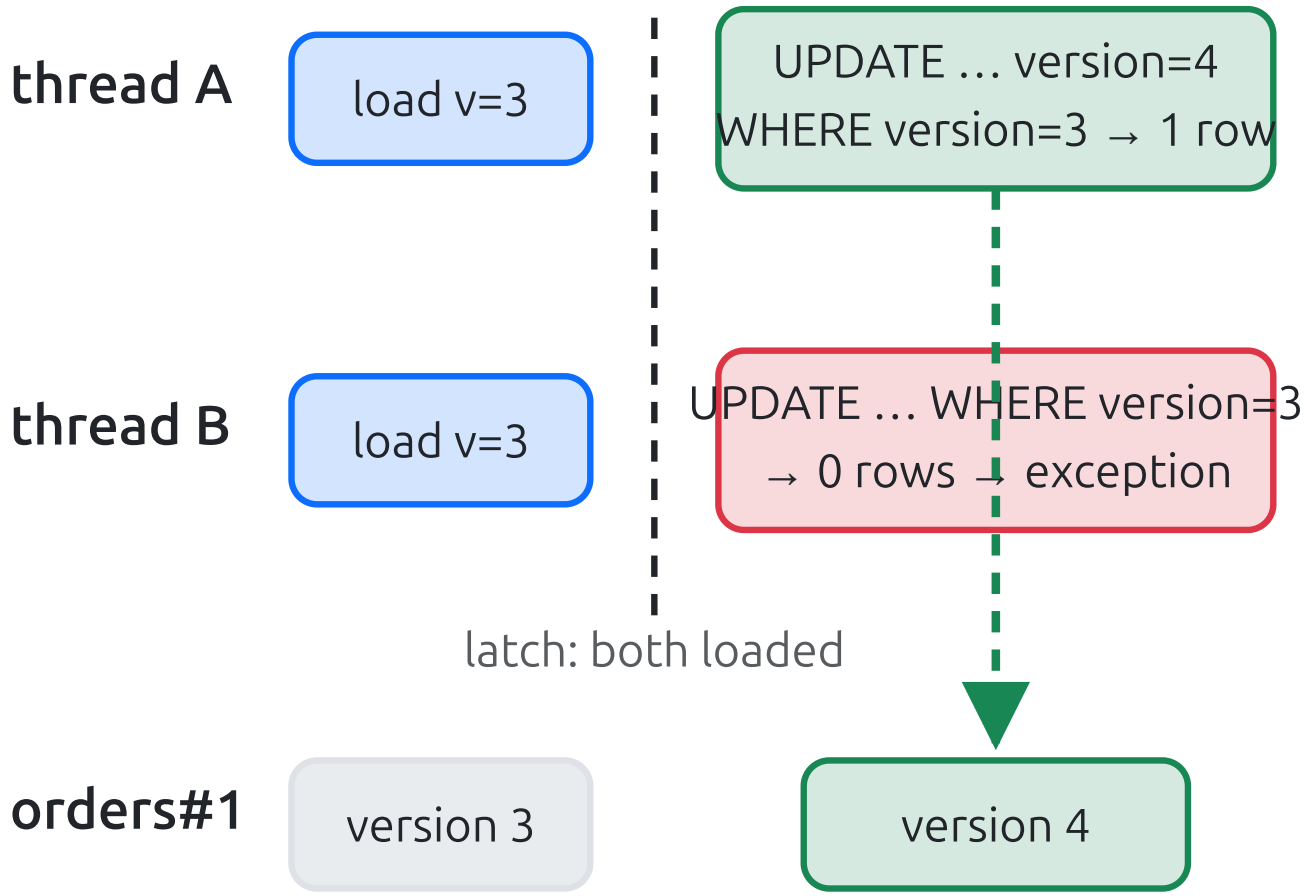
GOAL

Observe, with tests that read the statement log, what a managed entity does that a detached one does not; understand flush ordering; and prove with two real threads that `@Version` turns a lost update into an exception, then recover from it.

STEPS

1. Create `service/OrderService` with `@Transactional markPaid(Long orderId)`: load, `setStatus(PAID)`, no `save()`. In `OrderServiceTest` (`@SpringBootTest`) call it and assert (a) the status is PAID on reload and (b) exactly one `update orders` statement ran (use `Statistics.getEntityUpdateCount()` from `emf.unwrap(SessionFactory.class).getStatistics()`, cleared before the call).
2. The detached counter-example, in the test: `Product p = products.findById(id).orElseThrow()` (no transaction: `p` is detached at once), `p.setStock(99)`, reload in a new call and assert the stock is **unchanged**. Then `products.save(p)` and assert 99 is persisted; note in `README.md` that `save()` of a detached entity is a `merge`: one SELECT + one UPDATE in the log.

3. Add `@Transactional(readOnly = true) OrderDto view(Long id)`. Write a test that calls a deliberately wrong read-only method which modifies the entity, and show that nothing is flushed (Hibernate uses `FlushMode.MANUAL` under `readOnly`). Keep the test as documentation of the trap.
4. Flush order: in one `@Transactional` test method, `tags.delete(saleTag)` then `tags.save(new Tag("sale"))` and `flush()`. Expect a unique-constraint violation, because Hibernate executes INSERTs before DELETEs. Fix it with `tags.flush()` (or `em.flush()`) between the two operations and keep both variants as tests (one asserting the exception, one passing).
5. Optimistic locking: `OrderService.changeStatus(Long id, OrderStatus next)` (managed change, no save). Write `OptimisticLockingTest` (`@SpringBootTest`) with the starter below: two threads, each in its own `TransactionTemplate` transaction, both load the same order, wait on a `CountDownLatch` until both have loaded, then each sets a different status and commits. Assert that exactly one thread throws `ObjectOptimisticLockingFailureException`, the winner's status is stored and `version` increased by exactly 1.
6. Recovery: add `OrderFacade.changeStatusWithRetry` in a *separate* bean that catches `ObjectOptimisticLockingFailureException` and retries up to 3 times (loop, or Spring Framework 7 `@Retryable` with `@EnableResilientMethods`). Extend the test: with the facade both threads eventually succeed and the final version is the initial version + 2.
7. Enable `logging.level.org.springframework.orm.jpa=DEBUG` and `org.springframework.transaction.interceptor=TRACE` in `application-test.yml`; paste the lines showing "Getting transaction for [...OrderService.changeStatus]", the UPDATE with `version=?` and the rollback into `README.md`.



retry outside the failed tx: B reloads v=4 and commits v=5

STARTER CODE

```
package edu.itc.shop.service;

import java.util.List;
import java.util.Objects;
import java.util.concurrent.*;
import java.util.function.Function;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.transaction.support.TransactionTemplate;
import edu.itc.shop.domain.*;
import edu.itc.shop.repository.OrderRepository;
import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest
class OptimisticLockingTest {
```

```
    @Autowired OrderRepository orders;
    @Autowired TransactionTemplate tx;
    @Autowired TestData data;           // your helper: creates a customer, a
product, an order

    @Test
    void secondWriterFailsWithOptimisticLock() throws Exception {
        Long orderId = data.newOrder();
        long versionBefore =
orders.findById(orderId).orElseThrow().getVersion();
        CountDownLatch bothLoaded = new CountDownLatch(2);

        Function<OrderStatus, Callable<Throwable>> writer = target -> () -> {
            try {
                tx.executeWithoutResult(status -> {
                    Order o = orders.findById(orderId).orElseThrow();   // both
read the same version
                    bothLoaded.countDown();
                    await(bothLoaded);
                    o.setStatus(target);                                // dirty
→ UPDATE at commit
                });
                return null;
            } catch (RuntimeException e) { return e; }
        };

        try (ExecutorService pool = Executors.newFixedThreadPool(2)) {
            List<Future<Throwable>> results = pool.invokeAll(
                    List.of(writer.apply(OrderStatus.PAID),
writer.apply(OrderStatus.CANCELLED)));
            List<Throwable> failures =
results.stream().map(this::get).filter(Objects::nonNull).toList();

            assertThat(failures).hasSize(1);

assertThat(failures.get(0)).isInstanceOf(ObjectOptimisticLockingFailureException
.class);
            Order after = orders.findById(orderId).orElseThrow();
            assertThat(after.getVersion()).isEqualTo(versionBefore + 1);
            // TODO assert after.getStatus() is the status of the winning thread
        }
    }

    private static void await(CountDownLatch l) {
        try { if (!l.await(5, TimeUnit.SECONDS)) throw new
IllegalStateException("latch timeout"); }
        catch (InterruptedException e) { Thread.currentThread().interrupt();
throw new IllegalStateException(e); }
```

```
        }
        private Throwable get(Future<Throwable> f) {
            try { return f.get(); } catch (Exception e) { return e; }
        }
    }


    @Service
    public class OrderFacade {
        private final OrderService orderService;
        public OrderFacade(OrderService orderService) { this.orderService =
    orderService; }

        /** Retries the whole transaction; must live outside OrderService (the proxy
    raises at commit). */
        public void changeStatusWithRetry(Long id, OrderStatus next) {
            for (int attempt = 1; ; attempt++) {
                try { orderService.changeStatus(id, next); return; }
                catch (ObjectOptimisticLockingFailureException e) {
                    if (attempt == 3) throw e;
                    // TODO small back-off, then loop
                }
            }
        }
    }
```

EXPECTED OUTPUT

```
$ ./mvnw -q test -Dtest=OrderServiceTest,FlushOrderTest,OptimisticLockingTest
TRACE o.s.t.i.TransactionInterceptor : Getting transaction for
[edu.itc.shop.service.OrderService.changeStatus]
DEBUG org.hibernate.SQL : select
o1_0.id,o1_0.customer_id,o1_0.placed_at,o1_0.status,o1_0.version from orders
o1_0 where o1_0.id=?
DEBUG org.hibernate.SQL : select
o1_0.id,o1_0.customer_id,o1_0.placed_at,o1_0.status,o1_0.version from orders
o1_0 where o1_0.id=?
DEBUG org.hibernate.SQL : update orders set
customer_id=?,placed_at=?,status=?,version=? where id=? and version=?
DEBUG org.hibernate.SQL : update orders set
customer_id=?,placed_at=?,status=?,version=? where id=? and version=?
TRACE o.s.t.i.TransactionInterceptor : Completing transaction for
[...OrderService.changeStatus]
TRACE o.s.t.i.TransactionInterceptor : Completing transaction for
[...OrderService.changeStatus] after exception:
      org.springframework.orm.ObjectOptimisticLockingFailureException: Row was
updated or deleted by another
```

```
      transaction (or unsaved-value mapping was incorrect):
[edu.itc.shop.domain.Order#1]
...
# flush order without an intermediate flush():
org.springframework.dao.DataIntegrityViolationException: ... Unique index or
primary key violation:
      "PUBLIC.CONSTRAINT_INDEX_2 ON PUBLIC.TAG(NAME NULLS FIRST) VALUES ( /* 1
*/ 'sale' )"
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Spring Boot auto-configures a `TransactionTemplate` bean; use it instead of `@Transactional` on the test so that each thread gets its own transaction and the test method itself has none.
- Statistics counters are per `SessionFactory`: call `stats.clear()` right before the call under test, and keep the test class `@Execution(SAME_THREAD)`.
- If both threads succeed, they did not overlap: make sure the latch is awaited *inside* the transaction, after the load and before the commit.
- The `version` column must be `NOT NULL DEFAULT 0` in V1 and the field a primitive `long` (or `Long` with a starting value): a NULL version makes Hibernate treat the entity as new.
- For the flush-order experiment use the H2 constraint message as-is; on PostgreSQL the wording differs but the cause is the same: INSERT runs before DELETE inside one flush.

ACCEPTANCE CHECKLIST

- [ ] A managed change produces exactly one UPDATE without `save()`; a detached change produces nothing until `save()` (merge = SELECT + UPDATE); both are tested.
- [ ] The read-only trap and the flush-order constraint violation are reproduced by tests and explained in `README.md`, with the `flush()` fix.
- [ ] `OptimisticLockingTest` runs two real threads, exactly one gets `ObjectOptimisticLockingFailureException`, and the version increases by one.
- [ ] The retry lives in a separate bean, both writers eventually succeed, and the transaction/lock log excerpt is in `README.md`.

5 N+1, performance tuning and a second migration

Hard 75 min

GOAL

Reproduce an N+1 problem with a statement counter that fails the build, fix it three different ways and measure each, switch on JDBC batching, and evolve the schema with a Flyway `V2` migration that adds the indexes your queries need.

STEPS

1. Create `service/ReportService` with `@Transactional(readOnly = true) Map<Long, Integer> linesPerPaidOrder()` implemented naively: `orders.findByStatus(PAID)` (entity list) then `o.getLines().size()` for each. Create `test/…/QueryCounter`, a small helper around `emf.unwrap(SessionFactory.class).getStatistics()` exposing `reset()` and `count()` (`getPrepareStatementCount()`).
2. Write `NPlusOneTest` (`@SpringBootTest`): seed 50 PAID orders with 3 lines each through a `TestData` helper, then assert the naive implementation runs **51** statements. Keep this test (it documents the problem); mark it `@Tag("demo")`.
3. Fix A: `findByStatusWithLines` with `select distinct o from Order o join fetch o.lines where o.status = :s`. Fix B: `@EntityGraph(attributePaths = {"lines", "lines.product"}) List<Order> findWithGraphByStatus(OrderStatus)`. Fix C: keep the naive code but annotate `Order.lines` with `@BatchSize(size = 16)`. Add three methods to `ReportService` and three assertions: 1 statement, 1 statement, `1 + ceil(50/16) = 5` statements.
4. Measure: run each variant 20 times after a warm-up, record the median wall time in a table in `README.md` (columns: variant, statements, median ms). Then add the product name to the report (`line.getProduct().getName()`) and show which variants regress into N+1 again and how `spring.jpa.properties.hibernate.default_batch_fetch_size=32` caps it.
5. Batching: set `hibernate.jdbc.batch_size=50` and `order_inserts=true`. In `BatchInsertTest` save 1 000 products with `saveAll` in one transaction and assert `stats.getEntityInsertCount() == 1000` while `stats.getPrepareStatementCount()` is well below 1 000 (log `org.hibernate.engine.jdbc.batch=TRACE` shows "Executing batch"). Change the id strategy to `IDENTITY` in a scratch branch and observe the counter jump back to 1 000; note it in the README.
6. Confirm `open-in-view=false` matters: write `@WebMvcTest`-free evidence with a plain `@SpringBootTest` that calls a controller method (or the endpoint via `TestRestTemplate`) that touches `order.getLines()` outside the service transaction and assert the resulting `LazyInitializationException` / HTTP 500, then fix it by moving the mapping into the service.
7. Create `V2__indexes.sql`: `CREATE INDEX idx_orders_customer_placed ON orders (customer_id, placed_at DESC)`, `CREATE INDEX idx_order_line_order ON order_line (order_id)`, `CREATE INDEX idx_product_category_amount ON product (category, amount)`. Restart: Flyway reports "Migrating schema to version 2 - indexes"; `SELECT * FROM`

`flyway_schema_history` in the H2 console shows two rows. Run `EXPLAIN` on the `/api/orders` query in the console before and after and paste both plans into the README.

STARTER CODE

```
package edu.itc.shop.support;

import jakarta.persistence.EntityManagerFactory;
import org.hibernate.SessionFactory;
import org.hibernate.stat.Statistics;
import org.springframework.stereotype.Component;

/** Test helper: counts JDBC statements prepared by Hibernate since reset(). */
@Component
public class QueryCounter {
    private final Statistics stats;

    public QueryCounter(EntityManagerFactory emf) {
        this.stats = emf.unwrap(SessionFactory.class).getStatistics();
        stats.setStatisticsEnabled(true);
    }
    public void reset() { stats.clear(); }
    public long count() { return stats.getPrepareStatementCount(); }
    public long inserts() { return stats.getEntityInsertCount(); }
}


@SpringBootTest
@Execution(ExecutionMode.SAME_THREAD)
class NPlusOneTest {

    @Autowired ReportService report;
    @Autowired QueryCounter counter;
    @Autowired TestData data;

    @BeforeEach
    void seed() { data.paidOrders(50, 3); }      // 50 orders × 3 lines,
idempotent

    @Test @Tag("demo")
    void naiveImplementationRunsNPlusOneQueries() {
        counter.reset();
        report.linesPerPaidOrder();
        assertThat(counter.count()).isEqualTo(51);          // 1 + 50
    }

    @Test
    void joinFetchRunsOneQuery() {
```

```
        counter.reset();
        report.linesPerPaidOrderJoinFetch();
        assertThat(counter.count()).isEqualTo(1);
    }

    @Test
    void batchSizeRunsOnePlusCeilNOverBatch() {
        counter.reset();
        report.linesPerPaidOrderBatched();
        assertThat(counter.count()).isEqualTo(1 + (int) Math.ceil(50 / 16.0));
// 5
    }
    // TODO entityGraphRunsOneQuery()
}
```

```
# application.yml additions for Task 5
spring:
  jpa:
    properties:
      hibernate:
        jdbc.batch_size: 50
        order_inserts: true
        order_updates: true
        default_batch_fetch_size: 32
        generate_statistics: true
logging:
  level:
    org.hibernate.engine.jdbc.batch: TRACE
```

```
-- src/main/resources/db/migration/V2__indexes.sql
CREATE INDEX idx_orders_customer_placed  ON orders (customer_id, placed_at
DESC);
CREATE INDEX idx_order_line_order        ON order_line (order_id);
CREATE INDEX idx_product_category_amount ON product (category, amount);
```

EXPECTED OUTPUT

```
$ ./mvnw -q test -Dtest=NPlusOneTest,BatchInsertTest
DEBUG org.hibernate.SQL : select o1_0.id,... from orders o1_0 where
o1_0.status=?
DEBUG org.hibernate.SQL : select l1_0.order_id,l1_0.id,... from order_line l1_0
where l1_0.order_id=?
DEBUG org.hibernate.SQL : select l1_0.order_id,l1_0.id,... from order_line l1_0
where l1_0.order_id=?
... (48 more)
```

```
# with @BatchSize(16):
DEBUG org.hibernate.SQL : select l1_0.order_id,... from order_line l1_0 where
l1_0.order_id in (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
# batching:
TRACE o.h.e.jdbc.batch.internal.BatchImpl : Executing JDBC batch (50 / 50) -
insert into product ...
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0

# README.md (excerpt)
| variant                 | statements | median ms |
|-------------------------|-----------:|----------:|
| naive (lazy loop)       |         51 |      38.2 |
| join fetch              |          1 |       4.1 |
| @EntityGraph            |          1 |       4.4 |
| @BatchSize(16)          |          5 |       7.9 |

$ ./mvnw spring-boot:run
INFO  o.f.core.internal.command.DbMigrate : Migrating schema "PUBLIC" to version
"2 - indexes"
INFO  o.f.core.internal.command.DbMigrate : Successfully applied 1 migration to
schema "PUBLIC", now at version v2
```

Show hints

- Statistics count statements for the whole `SessionFactory`: reset immediately before the call and do not run tests in parallel.
- The fetch-join variant needs `Set` or exactly one `List` collection; fetching `lines` and `tags` together throws `MultipleBagFetchException`.
- `@BatchSize` is `org.hibernate.annotations.BatchSize`; the global `default_batch_fetch_size` applies to every lazy association and proxy, including `line.product`.
- Use `System.nanoTime()` and run in the same JVM; H2 in memory is fast, so the differences are small but the statement counts are the real evidence.
- Because the in-memory H2 database is recreated on every start, Flyway will apply V1 and V2 together; to see "now at version v2" alone, use a file URL (`jdbc:h2:file:./data/shop`) temporarily.

ACCEPTANCE CHECKLIST

- [ ] `NPlusOneTest` proves 51 statements for the naive code and 1 / 1 / 5 for join fetch, entity graph and `@BatchSize`.
- [ ] The README table with statement counts and median timings exists, including the "product name" regression and its fix.

- [ ] JDBC batching is enabled and proven by `BatchInsertTest`; the IDENTITY comparison is documented; `open-in-view=false` evidence is in the tests.
- [ ] `V2__indexes.sql` is applied by Flyway, V1 is untouched, and both `EXPLAIN` plans are in the README.

★ Challenge: reporting and bulk module Hard

120–150 min · optional, extra credit

SCENARIO

The shop's back office needs a product report with free combinations of filters, an order feed that scrolls forever without page numbers, a stock reservation that never oversells even under load, and a nightly import of the supplier catalogue (50 000 products) that must finish in seconds. Everything must be explained in a README that shows the schema and the SQL behind every query.

REQUIREMENTS

1. **Criteria API report**: `GET /api/reports/products?q=&category=&maxPrice=&minStock=&tag=&sort=` implemented in `ProductReportRepositoryImpl` (a custom fragment) with `CriteriaBuilder` directly: dynamic predicates, a join to `tags` only when `tag` is given, a whitelist of sortable attributes, and a record projection `ProductReportRow(sku, name, amount, stock, tagCount)` built with `cb.construct` and `groupBy`. Use the generated static metamodel (`Product_`) instead of strings.
2. **Infinite scroll**: `GET /api/orders/feed?size=20&after=<placedAt>,<id>` returns `{ items, nextCursor, hasNext }`. Implement it twice: with `Slice<OrderSummary>` and with keyset scrolling (`Window<Order>` + `ScrollPosition.keyset()`). Show in the README the SQL of both and explain why the keyset version stays fast at offset 100 000.
3. **Pessimistic stock reservation**: `StockService.reserve(productId, qty)` with `@Lock(LockModeType.PESSIMISTIC_WRITE)` on `ProductRepository.findForUpdate`. Test with 10 threads each reserving 1 unit from a product with `stock = 5`: exactly 5 succeed, 5 get `OutOfStockException`, stock ends at 0, never negative. Compare with the optimistic variant (`@Version` on `Product`) and with the single-statement `@Modifying update … where stock >= :qty`; record throughput for all three.
4. **Bulk import**: `BulkImportService.importCsv(Path)` reads a generated CSV of 50 000 products and inserts them with a Hibernate 7 `StatelessSession` (`sessionFactory.openStatelessSession()`, `setJdbcBatchSize(500)`, `insertMultiple`

per chunk of 500). Compare against `saveAll` in one `@Transactional` method (with and without `flush()/clear()` every 500). Report wall time and peak heap for the three runs. Target: under 10 s on H2.

5. **README**: a Mermaid `erDiagram` of the six tables (relationships and key columns), and a table listing every repository method in the project with the SQL Hibernate generates for it (copied from the log, parameters as `?`).

SKELETON

```
@Service
public class BulkImportService {
    private final SessionFactory sessionFactory;

    public BulkImportService(EntityManagerFactory emf) {
        this.sessionFactory = emf.unwrap(SessionFactory.class);
    }

    public int importCsv(Path csv) throws IOException {
        int imported = 0;
        try (StatelessSession ss = sessionFactory.openStatelessSession();
             Stream<String> lines = Files.lines(csv).skip(1)) {
            Transaction tx = ss.beginTransaction();
            ss.setJdbcBatchSize(500);
            List<Product> chunk = new ArrayList<>(500);
            for (Iterator<String> it = lines.iterator(); it.hasNext(); ) {
                chunk.add(parse(it.next()));
                if (chunk.size() == 500) { ss.insertMultiple(chunk); imported +=
chunk.size(); chunk.clear(); }
            }
            if (!chunk.isEmpty()) { ss.insertMultiple(chunk); imported +=
chunk.size(); }
            tx.commit();
        }
        return imported;
    }

    private Product parse(String line) { /*
sku;name;amount;currency;category;stock */ return null; }
}


erDiagram
    CUSTOMER ||--o{ ORDERS : places
    ORDERS ||--|{ ORDER_LINE : contains
    PRODUCT ||--o{ ORDER_LINE : "is sold as"
    PRODUCT ||--o{ PRODUCT_TAG : has
    TAG ||--o{ PRODUCT_TAG : labels
```

```
    CUSTOMER { bigint id PK  varchar email UK  varchar ship_city }
    ORDERS   { bigint id PK  bigint customer_id FK  varchar status  timestamptz
placed_at  bigint version }
    PRODUCT  { bigint id PK  varchar sku UK  decimal amount  char currency
varchar category  int stock }
```

EXPECTED OUTPUT

```
$ curl -s "localhost:8080/api/reports/products?
category=HARDWARE&minStock=1&tag=sale&sort=amount,desc" | jq '.[0]'
{ "sku": "KB-7", "name": "Mechanical keyboard", "amount": 129.00, "stock": 4,
"tagCount": 2 }

$ curl -s "localhost:8080/api/orders/feed?size=20" | jq '{n: (.items|length),
next: .nextCursor, hasNext}'
{ "n": 20, "next": "2026-09-20T08:11:02.331Z,1187", "hasNext": true }

$ ./mvnw -q test -Dtest=StockReservationTest,BulkImportTest
INFO  e.i.s.service.StockReservationTest : pessimistic: 5 ok / 5 out-of-stock,
stock=0, 10 threads in 212 ms
INFO  e.i.s.service.StockReservationTest : optimistic : 5 ok / 5 out-of-stock
after 17 retries, 486 ms
INFO  e.i.s.service.StockReservationTest : atomic upd : 5 ok / 5 out-of-stock,
61 ms
INFO  e.i.s.service.BulkImportTest       : stateless insertMultiple(500): 50000
rows in 3.9 s, 101 batches
INFO  e.i.s.service.BulkImportTest       : saveAll, one tx, no clear   : 50000
rows in 11.7 s, heap +610 MB
INFO  e.i.s.service.BulkImportTest       : saveAll + flush/clear(500)  : 50000
rows in 6.2 s
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Static metamodel: add `org.hibernate.orm:hibernate-processor` as an annotation processor in `maven-compiler-plugin`; `Product_` appears under `target/generated-sources/annotations`.
- The keyset feed needs a sort on `(placedAt desc, id desc)`; parse `after` into `ScrollPosition.of(Map.of("placedAt", t, "id", id), Direction.FORWARD)`.
- The reservation threads must each run in their own transaction (`TransactionTemplate`); a lock acquired without a transaction is released immediately.
- `StatelessSession` ignores cascades and the persistence context: set `Product.tags` aside for the import, and remember the sequence increment of 50 means Hibernate needs one `select next value` per 50 rows.
- For heap measurement use `Runtime.getRuntime().totalMemory() - freeMemory()` before and after, or run with `-Xmx256m` and watch which variant dies.

ACCEPTANCE CHECKLIST

- [ ] The Criteria report combines any subset of five filters, joins tags only when needed, sorts from a whitelist and returns a record projection with a metamodel-based implementation.
- [ ] The feed works with both `Slice` and keyset scrolling; the README shows both SQL statements and explains the difference.
- [ ] The 10-thread reservation test never oversells with the pessimistic lock; optimistic and atomic-update variants are measured and compared.
- [ ] 50 000 products import through `StatelessSession` in under 10 s with measurements for all three strategies; the README has the ER diagram and the method-to-SQL table.

Grading rubric

Component	Points	Full marks when…
Task 1 · Entities & mappings	10	Embeddable records, STRING enums, sequence ids; V1 matches the entities under `validate`; both mismatch experiments documented.
Task 2 · Associations & cascading	15	Correct owning sides, helpers, cascade/orphan removal proven by SQL log; hand-written identity methods and the identity test.
Task 3 · Repositories & queries	20	Derived, JPQL fetch join, native, projections, Specifications; paged endpoints; no entities leave the service layer.
Task 4 · States, dirty checking, locking	20	Managed vs detached and flush order shown by tests; two-thread optimistic lock test passes deterministically; retry outside the transaction.
Task 5 · Performance & migrations	25	N+1 reproduced and fixed three ways with statement counts and timings; batching proven; V2 indexes applied; EXPLAIN plans documented.
Code quality	10	Layered packages, `@Transactional` only in services, LAZY associations, DTOs at the boundary, wrapper committed, tags per task, no secrets.
★ Challenge (extra credit)	+20	All four acceptance items ticked; concurrency and import tests pass; README with ER diagram and SQL table.

Component	Points	Full marks when…
Total	**100 (+20)**	

Automatic deductions: −5 per `FetchType.EAGER` on a collection or unexplained eager `@ManyToOne`; −10 if `spring.jpa.hibernate.ddl-auto` is `update`/`create` in the main configuration or a migration file was edited after being applied; −5 per entity with Lombok `@Data`/`@EqualsAndHashCode`/`@ToString` or an `equals` based on mutable fields; −10 if `open-in-view` is left enabled or an entity is returned from a controller; −10 if `@Transactional` sits on a controller or a repository method instead of the service; −10 for committed passwords or `target/`.

Submission

1. Repository layout:

```
shop-jpa/
├── pom.xml  ·  mvnw  ·  .mvn/
├── README.md                ← schema-mismatch messages, SQL excerpts, N+1
table, EXPLAIN plans, challenge ER diagram
├── api-session.md           ← curl / .http transcript from Task 3
└── src/
    ├── main/java/edu/itc/shop/{domain,repository,service,web,web/dto}/…
    ├── main/resources/  application.yml · application-dev.yml ·
db/migration/V1__init.sql · V2__indexes.sql
    └── test/java/edu/itc/shop/…  ProductMappingTest · OrderCascadeTest ·
EntityIdentityTest · OrderRepositoryTest ·
                                  OrderServiceTest · FlushOrderTest ·
OptimisticLockingTest · NPlusOneTest · BatchInsertTest
```

2. `./mvnw -q verify` must succeed from a clean checkout on JDK 25 without environment variables (tests use H2 in memory and the Flyway migrations, never `create-drop`).
3. Tags `task-1` … `task-5` (and `challenge`) must exist. Push branch `lab-03` and open a merge request titled `Lab-03 – <your name>`.
4. Include a screenshot of the H2 console showing `flyway_schema_history` with both versions and one `EXPLAIN` result.

Back to Chapter 03 slides | Retake the chapter quiz | ← Lab-02